

# MASTER CODEX PROMPT: MULTI-TENANT EVENT TICKETING PLATFORM WITH USSD, MOBILE MONEY AND QR VALIDATION

Act as a senior software architect, senior Laravel developer, database architect, UI/UX developer, DevOps engineer and security engineer.

Build a complete, production-ready, multi-tenant event ticketing platform for Ghana. The platform will allow event organizers to create and manage events, sell tickets through the web, USSD, promoters, agents and box offices, collect payments through Mobile Money and cards, issue QR-code tickets, validate tickets at event gates, manage refunds and settle funds to organizers.

The system must be designed as a serious commercial SaaS product, not as a demo, prototype or tutorial.

The working product name is:

## **SmartCast Tickets**

The name must be configurable from the administration settings.

---

## 1. CORE OBJECTIVES

The system must support:

1. Multiple independent event organizers.
2. Public event discovery and ticket purchasing.
3. Guest checkout without mandatory account registration.
4. Phone-number-first customer identification.
5. OTP verification.
6. Web ticket purchases.
7. USSD ticket purchases.
8. Mobile Money payments.
9. Card payments.
10. QR-code ticket generation.
11. Human-readable ticket codes.
12. SMS ticket delivery.
13. Email ticket delivery.
14. Optional WhatsApp delivery architecture.
15. Online and offline gate validation.
16. Promoter and referral sales.
17. Box-office and agent sales.

18. Organizer commissions and settlement.
  19. Refund and ticket cancellation workflows.
  20. General-admission tickets.
  21. Reserved seating architecture.
  22. Complimentary and invitation tickets.
  23. Organizer team roles.
  24. Platform administration.
  25. Full audit logs.
  26. Financial ledger and reconciliation.
  27. Secure payment webhook handling.
  28. Responsive mobile-first design.
  29. REST API for scanner and external integrations.
  30. Local event validation architecture for unreliable internet environments.
- 

## 2. REQUIRED TECHNOLOGY STACK

Use the following stack:

### Backend

- Current stable Laravel release
- PHP 8.3 or newer
- MySQL 8
- Redis for cache, queues, locks and rate limiting
- Laravel queues
- Laravel Scheduler
- Laravel Events and Listeners
- Laravel Notifications
- Laravel Policies and Gates
- Laravel Form Requests
- Laravel API Resources
- Laravel Sanctum for API authentication
- Laravel Horizon where Redis is available

### Frontend

- Laravel Blade
- Bootstrap 5
- Alpine.js
- Livewire where it improves dashboard interactivity
- Vanilla JavaScript modules where appropriate
- DataTables only where necessary
- Chart.js for dashboard charts
- A dedicated JavaScript library for QR scanning
- Progressive Web App support for the scanner interface

Do not build the entire administration interface as a single-page React or Vue application.

React or Vue may only be introduced for highly interactive modules such as:

- Reserved seating map editor
- Complex seat selection
- Advanced scanner synchronization

## Supporting libraries

Use maintained packages for:

- QR-code generation
- PDF generation
- Spreadsheet export
- Role and permission management
- Two-factor or OTP support
- Image processing
- API documentation
- Activity logging

Before selecting a package, verify that it supports the Laravel and PHP versions used.

Do not use abandoned packages.

---

## 3. ARCHITECTURE

Build the application as a modular monolith.

Use clear domain boundaries rather than placing all logic inside controllers.

Recommended domain modules:

```
Authentication
Customers
Organizations
OrganizationKyc
Venues
Events
Ticketing
SeatManagement
Orders
Payments
USSD
```

```
Notifications
Promoters
Agents
GateManagement
Scanning
Refunds
Settlements
Ledger
Reporting
Support
Audit
Administration
PublicApi
```

Use this internal structure where appropriate:

```
app/
├─ Actions/
├─ Contracts/
├─ Data/
├─ Domain/
├─ Enums/
├─ Events/
├─ Exceptions/
├─ Http/
├─ Jobs/
├─ Listeners/
├─ Models/
├─ Notifications/
├─ Policies/
├─ Providers/
├─ Queries/
├─ Repositories/
├─ Rules/
├─ Services/
├─ Support/
└─ ValueObjects/
```

Controllers must remain thin.

Business logic must be placed in:

- Actions
- Services
- Domain classes

- Jobs
- Policies
- Dedicated query classes

Do not place payment, ticket issuance, settlement or inventory logic directly in controllers.

---

## 4. MULTI-TENANCY

Use a shared database with organization ownership columns.

Most organizer-owned records must have:

`organization_id`

All organizer queries must be scoped to the authenticated organization.

Prevent one organizer from viewing or modifying another organizer's:

- Events
- Orders
- Customers
- Tickets
- Settlements
- Promoters
- Agents
- Venues
- Reports
- Team members

Use:

- Global scopes where safe
- Policies
- Explicit organization checks
- Service-level authorization
- Automated multi-tenant security tests

Do not rely only on hidden form fields or frontend filtering.

Platform administrators may access all organizations.

---

## 5. USER TYPES AND ROLES

Implement these platform-level roles:

```
Super Admin
Platform Admin
Finance Administrator
Support Administrator
Compliance Officer
Technical Administrator
Report Viewer
```

Implement these organizer roles:

```
Organization Owner
Organization Administrator
Event Manager
Finance Officer
Ticket Manager
Marketing Officer
Promoter Manager
Box Office Supervisor
Box Office Agent
Gate Supervisor
Scanner Operator
Report Viewer
```

Use granular permissions.

Examples:

```
organizations.view
organizations.approve
organizations.suspend
events.create
events.update
events.publish
events.cancel
tickets.create
tickets.issue_complimentary
orders.view
payments.view
refunds.request
```

```
refunds.approve
settlements.request
settlements.approve
promoters.manage
agents.manage
gates.manage
scans.perform
scans.reverse
reports.financial
reports.attendance
```

A scanner operator must not see financial reports.

A promoter must only see their attributed sales and commissions.

A finance officer must not automatically have event publishing permission.

---

## 6. AUTHENTICATION

Support:

1. Email and password login for staff.
2. Phone number and OTP login for customers.
3. Password reset.
4. Email verification.
5. Optional two-factor authentication for administrators.
6. Session management.
7. Login history.
8. Device and IP logging.
9. Account lockout and rate limiting.
10. Optional trusted devices.

Customer accounts should not be required before checkout.

After a guest checkout, allow the customer to claim their account using OTP.

Normalize Ghanaian phone numbers into E.164 format.

Examples:

```
0241234567 → +233241234567
233241234567 → +233241234567
+233241234567 → +233241234567
```

Create a reusable phone-number value object and validation rule.

---

## 7. ORGANIZER ONBOARDING AND KYC

Organizations must register and submit:

- Organization name
- Business or individual type
- Contact person
- Phone number
- Email
- Address
- GhanaPost GPS address
- Registration number where applicable
- Tax identification number where applicable
- Bank account details
- Mobile Money settlement details
- Identification documents
- Business registration documents
- Logo
- Website and social links

KYC statuses:

```
draft
submitted
under_review
approved
rejected
additional_information_required
suspended
```

The system must maintain:

- Reviewer
- Review date
- Review notes
- Rejection reasons
- Uploaded document history
- Approval history

An organizer must not publish paid events or request settlements until approved.

---

## 8. PUBLIC EVENT WEBSITE

Create a professional public frontend with:

- Home page
- Featured events
- Upcoming events
- Event categories
- Event search
- Event filtering
- Organizer pages
- Event detail page
- Ticket selection
- Checkout
- Payment status
- Ticket retrieval
- Customer ticket wallet
- Refund request
- Ticket transfer
- Terms and conditions
- Privacy policy
- Contact and support pages

Event filters should include:

- Location
- Date
- Category
- Free or paid
- Online or physical
- Price range
- Organizer

Each event should have a public slug.

Example:

`/events/breman-odwira-concert-2026`

Provide social sharing metadata and Open Graph images.

---

## 9. EVENT MANAGEMENT

An event must support:

- Organization
- Event title
- Slug
- Short event code
- Description
- Short description
- Event category
- Event format
- Venue
- Address
- GPS coordinates
- Start date and time
- End date and time
- Time zone
- Doors-open time
- Sales start
- Sales end
- Banner
- Thumbnail
- Gallery
- Age restriction
- Dress code
- Terms
- Refund policy
- Transfer policy
- Visibility
- Publication status
- Featured status
- Capacity
- Currency
- Contact details
- Custom questions
- SEO metadata

Event statuses:

```
draft
pending_review
published
sales_paused
sold_out
postponed
```

cancelled  
completed  
archived

Event formats:

physical  
virtual  
hybrid

Support events with multiple sessions or dates.

Examples:

- A conference running for three days
- A cinema showing at different times
- A festival with morning and evening sessions
- A recurring weekly programme

Create an `event_sessions` table.

---

## 10. TICKET TYPES

Support:

General Admission  
Regular  
VIP  
VVIP  
Student  
Early Bird  
Late Entry  
Couple  
Group  
Family  
Table  
Booth  
Reserved Seat  
Complimentary  
Media  
Staff  
Sponsor

Backstage  
Custom

Each ticket type must support:

- Event or event session
- Name
- Description
- Price
- Original price
- Quantity
- Minimum quantity per order
- Maximum quantity per order
- Sales start
- Sales end
- Visibility
- Refundability
- Transferability
- Ticket color
- Ticket access zone
- Gate restrictions
- Tax configuration
- Service fee configuration
- Status

Ticket type statuses:

draft  
active  
paused  
sold\_out  
expired  
archived

Support hidden ticket types accessible only through a private URL or promo code.

---

## 11. INVENTORY AND RESERVATION

Ticket inventory must be concurrency safe.

Do not calculate availability using only:

```
quantity - count(orders)
```

Use dedicated inventory and reservation logic.

Inventory must track:

```
total_quantity  
reserved_quantity  
sold_quantity  
complimentary_quantity  
blocked_quantity  
available_quantity
```

Use database transactions and row-level locking.

When a customer starts checkout:

1. Lock inventory.
2. Confirm availability.
3. Create an inventory reservation.
4. Set an expiration time.
5. Release expired reservations through a scheduled job.
6. Convert reservation to sold inventory after confirmed payment.

Reservation statuses:

```
active  
converted  
expired  
cancelled  
released
```

Use Redis locks where helpful, but the database must remain the final authority.

Prevent overselling even when multiple payment callbacks arrive simultaneously.

---

## 12. RESERVED SEATING

Design the database and services to support:

- Venue sections
- Rows

- Seats
- Tables
- Booths
- Standing zones
- Disabled-access seats
- Seat pricing zones
- Seat blocking
- Seat holds
- Seat maps

Seat statuses:

```
available  
held  
sold  
blocked  
reserved  
unavailable
```

The initial release may provide general admission first, but reserved seating architecture must be included.

Create support for:

- Reusable venue seating templates
- Event-specific seat overrides
- Different prices by section
- Seat selection locks
- Hold expiration
- Administrative seat blocking
- Complimentary reserved seats

Never trust the seat status shown in the browser. Revalidate on the server before creating an order.

---

## 13. CHECKOUT

Guest checkout must request only necessary details.

Required:

- Phone number
- First name
- Last name

Optional depending on organizer settings:

- Email
- Gender
- Age range
- Address
- Organization
- Custom event questions

Checkout flow:

1. Select event session.
2. Select ticket type or seat.
3. Select quantity.
4. Add attendee details if required.
5. Apply discount code.
6. Display fees.
7. Accept terms.
8. Select payment method.
9. Create order.
10. Initiate payment.
11. Confirm payment asynchronously.
12. Issue tickets.
13. Deliver tickets.

Use a checkout session model so users can resume interrupted checkout.

Never mark an order paid based on a frontend redirect.

---

## 14. ORDER MANAGEMENT

Order statuses:

```
draft
reserved
pending_payment
payment_processing
paid
partially_paid
payment_failed
expired
cancelled
partially_refunded
refunded
```

disputed  
chargeback

Each order must include:

- Public order reference
- Organization
- Event
- Event session
- Customer
- Buyer phone
- Buyer email
- Currency
- Subtotal
- Discount
- Service fee
- Gateway fee
- Tax
- Total
- Amount paid
- Amount refunded
- Sales channel
- Promoter
- Agent
- Checkout session
- Payment status
- IP address
- Device information
- UTM attribution
- Notes

Sales channels:

web  
mobile\_web  
ussd  
organizer\_dashboard  
promoter\_link  
agent\_pos  
box\_office  
complimentary  
api\_partner  
admin

Order items must preserve a price snapshot.

Do not rely on the current ticket-type price after purchase.

---

## 15. PAYMENT ARCHITECTURE

Create a provider-independent payment system.

Define a contract similar to:

```
interface PaymentGatewayContract
{
    public function initiate(PaymentInitiationData $data):
    PaymentInitiationResult;

    public function verify(string $providerReference):
    PaymentVerificationResult;

    public function refund(PaymentRefundData $data): PaymentRefundResult;

    public function validateWebhook(
        array $payload,
        array $headers,
        string $rawBody
    ): bool;

    public function parseWebhook(
        array $payload,
        array $headers,
        string $rawBody
    ): PaymentWebhookData;
}
```

Implement provider adapters:

```
HubtelPaymentGateway
PaystackPaymentGateway
MtnMomoPaymentGateway
ManualPaymentGateway
FakePaymentGateway
```

The Fake gateway must only be available in local and testing environments.

Payment methods:

```
MTN Mobile Money
Telecel Cash
AT Money
Visa
Mastercard
GHQR
Bank Transfer
Cash
POS
Organizer Wallet
Complimentary
```

Payment statuses:

```
created
pending
processing
successful
failed
cancelled
expired
reversed
partially_refunded
refunded
disputed
chargeback
```

Store every payment attempt separately.

A single order may have multiple failed attempts before one succeeds.

---

## 16. PAYMENT WEBHOOKS

Webhook handling is security-critical.

For every webhook:

1. Capture the raw request body.
2. Capture headers.
3. Validate provider signature.
4. Store the original payload.
5. Assign an idempotency key.
6. Reject invalid signatures.

7. Respond quickly.
8. Queue processing.
9. Verify payment with the provider when necessary.
10. Process safely even when the same webhook is received multiple times.
11. Prevent duplicate ticket issuance.
12. Record all state changes.

Create a `payment_webhooks` table with:

```
provider
event_type
provider_reference
idempotency_key
signature_valid
headers
payload
raw_body
received_at
processed_at
processing_status
failure_reason
attempts
```

Never store Mobile Money PINs, card numbers, CVV or sensitive card data.

---

## 17. TICKET ISSUANCE

Tickets must only be activated after server-confirmed successful payment, except for authorized complimentary or cash workflows.

Every ticket must contain:

- ULID or UUID
- Public ticket reference
- Short human-readable code
- Secure random token
- Signed QR payload
- Order item
- Event
- Event session
- Ticket type
- Customer or attendee
- Price snapshot
- Status

- Issued date
- Validity period
- Transfer status
- Check-in status
- Current owner
- Original owner
- Sales channel

Ticket statuses:

```
reserved
pending
active
checked_in
transferred
cancelled
refunded
voided
expired
revoked
```

The QR code must not expose a simple sequential database ID.

Use:

- Cryptographically random tokens
- Signed payloads
- Key rotation support
- Token versioning

Example ticket reference:

```
OD26-VIP-7K4M9Q
```

Generate:

- Mobile ticket page
- QR-code image
- Printable PDF ticket
- Apple/Google wallet architecture placeholder where appropriate

Do not generate duplicate tickets when the ticket-issuance job is retried.

---

## 18. TICKET DELIVERY

Support:

- SMS
- Email
- Downloadable PDF
- Public mobile ticket link
- Customer ticket wallet
- Optional WhatsApp adapter

Create provider-independent notification contracts.

SMS providers may include:

```
HubtelSmsProvider
ArkeselSmsProvider
MnotifySmsProvider
FakeSmsProvider
```

Notification statuses:

```
queued
processing
sent
delivered
failed
cancelled
```

Store:

- Channel
- Recipient
- Template
- Provider
- Provider reference
- Cost
- Attempts
- Failure reason
- Sent time
- Delivery time

Create reusable message templates editable by administrators.

---

## 19. USSD MODULE

USSD must be treated as a major sales channel.

Create a provider-independent USSD gateway contract.

USSD flow:

Welcome to SmartCast Tickets

1. Buy Ticket
2. Retrieve Ticket
3. Check Payment
4. Transfer Ticket
5. Help

Buy-ticket flow:

1. Browse Featured Events
2. Enter Event Code

After event selection:

- Select Ticket Type
1. Regular - GHS 50
  2. VIP - GHS 100
  3. VVIP - GHS 250

Then:

Enter Quantity

Then display:

VIP x 2  
Ticket Amount: GHS 200  
Service Fee: GHS 5  
Total: GHS 205

1. Confirm
2. Cancel

After confirmation:

1. Create inventory reservation.
2. Create order.
3. Initiate Mobile Money collection.
4. Return an appropriate USSD response.
5. Send payment-status SMS.
6. Issue ticket only after payment confirmation.
7. Send ticket codes and mobile ticket link by SMS.

The USSD module must support:

- Shared shortcode
- Event-specific extension
- Short event codes
- Session timeout
- Session resume where possible
- Network detection
- Phone number normalization
- Pagination
- Localization architecture
- Error recovery
- Payment-status checks
- Ticket retrieval
- Quantity limits
- Sold-out handling
- Expired-session handling

Create a state-machine design rather than a large nested switch statement.

Recommended session states:

```
welcome
buy_method
browse_events
enter_event_code
select_event
select_session
select_ticket_type
enter_quantity
confirm_order
payment_initiated
payment_pending
payment_successful
```

```
payment_failed
retrieve_ticket
check_payment
transfer_ticket
help
completed
cancelled
expired
```

Create `ussd_sessions` with:

```
id
provider
provider_session_id
phone_number
network
current_state
previous_state
event_id
event_session_id
ticket_type_id
quantity
order_id
payment_id
session_data
started_at
last_activity_at
expires_at
status
```

Store session state in the database or Redis with database persistence.

Do not depend on normal Laravel web sessions for USSD.

Create a simulator page for local development where developers can test USSD flows without a live shortcode.

---

## 20. TICKET RETRIEVAL

Customers must be able to retrieve tickets using:

- Phone number and OTP
- Order reference

- Ticket code
- Email
- USSD
- Support-assisted lookup

Never reveal all tickets after entering only a phone number.

Require OTP or another verification factor.

Allow gate staff to search by phone number only when they have gate authorization.

---

## 21. TICKET TRANSFER

Ticket transfer flow:

1. Current owner enters recipient phone number or email.
2. Validate transfer policy.
3. Send OTP or acceptance link.
4. Recipient accepts.
5. Revoke old QR token.
6. Generate new QR token.
7. Change ownership.
8. Record transfer history.
9. Notify both parties.

Transfer statuses:

```
requested
pending_acceptance
accepted
rejected
expired
cancelled
```

Organizers must be able to:

- Disable transfers
- Set transfer deadline
- Limit transfer count
- Require transfer fee
- Approve transfers manually

The previous QR code must become invalid immediately after a transfer.

---

## 22. REFUNDS

Refund workflow:

```
requested
under_review
approved
rejected
submitted_to_gateway
processing
completed
failed
cancelled
```

Support:

- Full order refund
- Partial order refund
- Individual ticket refund
- Service-fee refund settings
- Organizer approval
- Platform approval
- Automatic refund after event cancellation
- Manual refund
- Gateway refund
- Refund notes
- Supporting documents

After a successful refund:

- Revoke relevant ticket
- Update order totals
- Update ledger
- Reverse promoter commission where applicable
- Reverse organizer payable balance
- Record audit trail
- Return inventory when policy permits

---

## 23. PROMOTERS AND REFERRALS

Promoters must have:

- Profile

- Organizer association
- Unique referral code
- Unique referral link
- Optional promo code
- Commission type
- Commission value
- Sales target
- Active dates
- Event assignments
- Dashboard
- Sales report
- Commission balance
- Payout history

Commission types:

```
percentage
fixed_per_ticket
fixed_per_order
tiered
```

Attribution rules:

1. Capture referral code at first event-page visit.
2. Store it in checkout session.
3. Persist it on the order.
4. Do not depend only on cookies.
5. Allow organization-defined attribution expiration.
6. Prevent self-referral abuse.
7. Preserve audit history.

Promoters must not have access to customer-sensitive details beyond what is necessary.

---

## 24. AGENTS AND BOX-OFFICE SALES

Agents should be able to:

- Sell tickets in person
- Accept cash
- Initiate Mobile Money payment
- Record POS payment
- Send tickets by SMS
- Print tickets
- View shift sales
- Open and close shifts

- Reconcile cash
- Void transactions with permission
- Reprint tickets
- Search transactions

Create:

```
agents
agent_devices
agent_shifts
agent_cash_movements
agent_ticket_allocations
agent_reconciliations
```

For cash sales, support:

- Prefunded allocation
- Credit limit
- Daily sales limit
- Supervisor approval
- Cash reconciliation
- Variance recording

An agent must not be able to issue unlimited unpaid tickets.

---

## 25. GATE MANAGEMENT

Each event may have multiple gates.

A gate should support:

- Name
- Description
- Event
- Location
- Allowed ticket types
- Allowed zones
- Opening time
- Closing time
- Capacity
- Assigned supervisors
- Assigned scanner devices

Examples:

Main Gate  
VIP Gate  
Backstage Gate  
Staff Entrance  
North Gate  
Table Entrance

## 26. SCANNER APPLICATION

Build a mobile-first PWA scanner.

Scanner functionality:

- Camera QR scan
- Manual ticket-code entry
- Search by phone number
- Search by order reference
- Ticket validation
- Check-in
- Duplicate-scan warning
- Wrong-event warning
- Wrong-gate warning
- Revoked-ticket warning
- Refunded-ticket warning
- Offline mode
- Synchronization
- Sound feedback
- Vibration feedback
- Flashlight control
- Scan history
- Device registration
- Operator login
- Gate assignment

Validation result states:

VALID  
ALREADY CHECKED IN  
INVALID  
WRONG EVENT  
WRONG SESSION  
WRONG GATE  
REFUNDED

REVOKED  
EXPIRED  
TRANSFERRED  
OFFLINE UNVERIFIED

Display large, clear screens for gate staff.

Do not expose unnecessary financial information to scanner operators.

---

## 27. OFFLINE SCANNING

Implement offline-capable scanning.

Before an event:

1. Authorized device downloads a signed event package.
2. Package contains valid ticket token fingerprints.
3. Package contains revoked tokens.
4. Package contains ticket-type and gate rules.
5. Package has version and expiration.
6. Package is stored in IndexedDB.
7. Device records scans locally.
8. Device synchronizes when connectivity returns.

Offline scan records must include:

- Local scan UUID
- Device
- Operator
- Gate
- Ticket token
- Local timestamp
- Server-received timestamp
- Offline status
- Sync status
- Conflict status

Conflict types:

```
duplicate_on_another_device  
revoked_after_download  
wrong_gate  
already_checked_in_online
```

```
invalid_package
stale_package
```

Create synchronization endpoints with idempotency.

Document the unavoidable duplicate risk when multiple devices are completely offline.

Design an optional local venue server architecture for later implementation.

---

## 28. LOCAL EVENT SERVER ARCHITECTURE

Provide an optional local validation mode.

Architecture:

```
Cloud Application
  ↓
Local Event Server
  ↓
Venue Wi-Fi LAN
  ↓
Scanner Devices
```

The local server should:

- Download event ticket data before the event
- Validate scanners over LAN
- Coordinate check-ins centrally
- Operate without internet
- Synchronize with cloud when internet returns
- Maintain encrypted local data
- Reject unauthorized devices

The first version may include architecture, interfaces and documentation without requiring full hardware deployment.

---

## 29. FINANCIAL LEDGER

Do not calculate balances using only order totals.

Build an immutable ledger system.

Recommended ledger accounts:

Gateway Clearing  
Platform Cash  
Organizer Payable  
Platform Commission Revenue  
Service Fee Revenue  
SMS Revenue  
USSD Revenue  
Promoter Commission Payable  
Agent Commission Payable  
Refund Liability  
Chargeback Liability  
Tax Payable  
Settlement Clearing

Ledger entries must be:

- Immutable
- Balanced
- Referenced to source transactions
- Timestamped
- Auditable

Support debit and credit entries.

Every financial event must create appropriate ledger transactions:

- Successful payment
- Refund
- Commission recognition
- Organizer settlement
- Promoter commission
- Agent commission
- Gateway fee
- Service fee
- Chargeback
- Manual adjustment

Manual adjustments require:

- Authorized permission
- Reason
- Supporting notes
- Audit log
- Optional second approval

Use decimal monetary fields, never floating-point types.

---

## 30. SETTLEMENTS

Organizer settlement workflow:

```
calculated
available
requested
under_review
approved
processing
paid
failed
rejected
cancelled
reversed
```

Settlement calculation must show:

```
Gross sales
Discounts
Refunds
Chargebacks
Payment gateway fees
Platform commission
Service fees
SMS charges
USSD charges
Promoter commissions
Agent commissions
Tax deductions
Previous adjustments
Net settlement
```

Support:

- Event-based settlement
- Periodic settlement
- Partial settlement
- Settlement hold
- Reserve balance
- Minimum payout

- Bank transfer
- Mobile Money payout
- Manual payout
- Payout proof
- Settlement statement PDF

Prevent settlement of disputed, refunded or unavailable funds.

---

## 31. PLATFORM PRICING AND COMMISSION

Allow administrators to configure:

- Platform commission percentage
- Flat platform fee
- Buyer-paid service fee
- Organizer-paid service fee
- Gateway-fee handling
- SMS markup
- USSD markup
- Promoter commission rules
- Agent commission rules
- Tax
- Subscription package
- Event-specific override
- Organization-specific override

Pricing hierarchy:

```
Event override
Organization override
Subscription package
Global default
```

Maintain effective dates and history.

Do not retroactively change existing order calculations.

---

## 32. DISCOUNTS AND PROMO CODES

Support:

- Percentage discounts

- Fixed discounts
- Free tickets
- Buy-one-get-one
- Quantity-based discounts
- Early-bird pricing
- Group pricing
- Promoter codes
- Private access codes

Promo code restrictions:

- Event
- Session
- Ticket type
- Minimum order
- Maximum discount
- Usage limit
- Usage per customer
- Start and end date
- Phone-number whitelist
- Email-domain restriction

Prevent race conditions around usage limits.

---

## 33. REPORTING

Create reports for:

### Platform

- Total gross ticket sales
- Platform revenue
- Gateway fees
- Service fees
- Organizer liabilities
- Settlements
- Refunds
- Chargebacks
- Active organizers
- Active events
- Payment conversion
- Sales by channel
- USSD sales
- Agent sales
- Promoter sales

- SMS usage
- Scan activity
- Fraud alerts

## Organizer

- Sales by event
- Sales by ticket type
- Sales by date
- Sales by channel
- Payment methods
- Promoter performance
- Agent performance
- Attendance
- Check-in time
- Gate traffic
- Refunds
- Settlement status
- Customer export
- Ticket status
- Revenue breakdown

Reports should support:

- Date filters
- Event filters
- Ticket filters
- Channel filters
- Export to CSV
- Export to Excel
- Export to PDF
- Print view

Use queued exports for large reports.

---

## 34. DASHBOARDS

### Super Admin dashboard

Display:

- Gross ticket sales
- Platform revenue
- Pending organizer approvals
- Pending settlements

- Failed payments
- Refund requests
- Current live events
- Tickets sold today
- USSD transactions
- Scanner activity
- System alerts

## Organizer dashboard

Display:

- Total sales
- Tickets sold
- Available tickets
- Upcoming events
- Live event attendance
- Revenue by event
- Sales by channel
- Promoter leaderboard
- Settlement balance
- Recent orders
- Refund alerts

## Scanner dashboard

Display:

- Event
- Assigned gate
- Online or offline status
- Total scans
- Valid scans
- Duplicate scans
- Invalid scans
- Last synchronization

All dashboards must be responsive.

---

# 35. ADMINISTRATION SETTINGS

Create settings for:

- Platform name
- Logo

- Favicon
- Contact information
- Currency
- Time zone
- Date format
- Phone country code
- Email provider
- SMS provider
- USSD provider
- Payment gateways
- Ticket code format
- QR signing keys
- Reservation duration
- Settlement rules
- Service fees
- Commission
- Tax
- Terms
- Privacy policy
- Refund defaults
- Support information
- Maintenance mode
- Branding
- Social media links

Encrypt sensitive configuration values.

Do not store secrets directly in source-controlled files.

---

## 36. CUSTOMER SUPPORT AND DISPUTES

Create support tickets with:

- Category
- Priority
- Customer
- Organizer
- Order
- Payment
- Event
- Assigned staff
- Status
- Messages
- Attachments
- Internal notes

Statuses:

```
open
pending_customer
pending_organizer
pending_payment_provider
under_investigation
resolved
closed
```

Support disputes for:

- Payment completed but ticket missing
- Duplicate debit
- Wrong ticket
- Event cancelled
- Refund delayed
- Ticket transfer problem
- Invalid ticket
- Gate denial

---

## 37. AUDIT LOGGING

Record all important operations:

- Login
- Failed login
- Account changes
- Organizer approval
- Event publication
- Ticket-type changes
- Ticket issuance
- Complimentary ticket issuance
- Manual payment confirmation
- Refund approval
- Settlement approval
- Scan reversal
- Permission change
- Financial adjustment
- Configuration change
- API credential changes

Audit logs must store:

- User
- Organization
- Action
- Entity type
- Entity ID
- Old values
- New values
- IP address
- User agent
- Timestamp
- Request correlation ID

Audit logs must not be editable by normal users.

---

## 38. SECURITY REQUIREMENTS

Implement:

- CSRF protection
- XSS protection
- SQL injection protection
- Strong input validation
- Output escaping
- Secure cookies
- HTTPS enforcement in production
- Rate limiting
- OTP rate limiting
- Login lockout
- API token expiration
- Role and permission checks
- Tenant isolation
- Webhook signature validation
- Idempotency
- Signed URLs
- Secure file uploads
- MIME validation
- Malware-scanning integration point
- Secret encryption
- Sensitive-field encryption
- Security headers
- Content Security Policy
- Clickjacking protection
- Request correlation IDs

- Centralized exception handling
- Sensitive-data masking in logs

Do not log:

- Mobile Money PINs
- Card numbers
- CVV
- Passwords
- OTP values
- Full payment authorization secrets

Use secure random generators for:

- Ticket tokens
- OTPs
- API keys
- Public references

Create security tests for:

- Cross-tenant access
- Unauthorized refunds
- Unauthorized settlements
- Forged webhooks
- Duplicate webhook processing
- Ticket-code guessing
- QR tampering
- Scanner access
- Expired OTPs
- Rate limits

---

## 39. DATA PROTECTION

Create technical support for:

- Privacy consent
- Marketing consent
- Consent history
- Data export request
- Data deletion request
- Data-retention policy
- Customer anonymization
- Organizer data-processing agreement
- Restricted staff access
- Personal-data audit

Do not delete financial transaction records that must be retained. Anonymize customer identity where legally and operationally appropriate.

---

## 40. DATABASE REQUIREMENTS

Create proper Laravel migrations, foreign keys and indexes.

Core tables should include:

```
users
customers
customer_devices
otp_requests
organizations
organization_users
organization_kyc_submissions
organization_documents
venues
venue_sections
venue_rows
venue_seats
events
event_sessions
event_images
event_questions
ticket_types
ticket_inventory
seat_holds
inventory_reservations
checkout_sessions
orders
order_items
order_attendees
payments
payment_attempts
payment_webhooks
tickets
ticket_tokens
ticket_transfers
ticket_scans
gates
gate_ticket_types
scanner_devices
scanner_sync_batches
```

```
promoters
promoter_event_assignments
promoter_attributions
promoter_commissions
agents
agent_devices
agent_shifts
agent_cash_movements
agent_reconciliations
discount_codes
discount_code_redemptions
refunds
refund_items
settlements
settlement_items
ledger_accounts
ledger_transactions
ledger_entries
ussd_sessions
notifications
notification_templates
support_tickets
support_messages
audit_logs
settings
api_clients
webhook_endpoints
webhook_deliveries
```

Use:

- Foreign keys
- Unique constraints
- Composite indexes
- Appropriate nullable fields
- Decimal money fields
- JSON only for flexible metadata, not for core relational data
- Soft deletes only where appropriate
- ULIDs or UUIDs for public identifiers
- Timestamps
- Explicit status fields using enums or constrained values

Add indexes for:

- organization\_id
- event\_id
- order reference

- payment reference
- ticket code
- ticket token hash
- customer phone
- USSD session ID
- scan timestamps
- settlement status
- webhook idempotency key

---

## 41. ENUMS AND STATE MACHINES

Use PHP backed enums for stable statuses.

Do not scatter string status values throughout the application.

Create state transition services for:

- Event status
- Order status
- Payment status
- Ticket status
- Refund status
- Settlement status
- Transfer status
- KYC status
- USSD session status

Reject illegal transitions.

Example:

```
pending_payment → paid
pending_payment → expired
pending_payment → cancelled

paid must not transition directly to draft.
```

State changes must create audit records and domain events.

---

## 42. BACKGROUND JOBS

Use queues for:

- Payment webhook processing
- Payment verification
- Ticket generation
- PDF generation
- QR generation
- SMS sending
- Email sending
- WhatsApp sending
- Inventory-release jobs
- Settlement calculation
- Report exports
- Scanner synchronization
- Notification retries
- Webhook delivery
- Fraud analysis
- Event reminders

All jobs must be idempotent.

Configure retry policies and failed-job monitoring.

---

## 43. SCHEDULED TASKS

Schedule tasks for:

- Release expired reservations
- Expire checkout sessions
- Verify pending payments
- Retry failed notifications
- Recalculate event sold-out status
- Generate settlement availability
- Send event reminders
- Expire transfer requests
- Rotate offline scanner packages
- Clean temporary uploads
- Monitor failed webhooks
- Archive old sessions
- Generate daily reconciliation reports

Use distributed locks so scheduled tasks do not run twice across multiple servers.

---

## 44. API DESIGN

Use versioned APIs:

```
/api/v1/
```

Suggested endpoints:

```
POST /api/v1/auth/otp/request
POST /api/v1/auth/otp/verify

GET /api/v1/events
GET /api/v1/events/{event}
GET /api/v1/events/{event}/inventory

POST /api/v1/checkouts
POST /api/v1/checkouts/{checkout}/reserve
POST /api/v1/orders
GET /api/v1/orders/{reference}

POST /api/v1/payments/initiate
GET /api/v1/payments/{reference}
POST /api/v1/payments/{payment}/verify

POST /api/v1/webhooks/hubtel
POST /api/v1/webhooks/paystack
POST /api/v1/webhooks/mtn-momo

GET /api/v1/tickets
GET /api/v1/tickets/{ticket}
POST /api/v1/tickets/{ticket}/transfer
POST /api/v1/tickets/retrieve

POST /api/v1/scanner/auth
POST /api/v1/scanner/packages/download
POST /api/v1/tickets/validate
POST /api/v1/tickets/check-in
POST /api/v1/tickets/check-in/reverse
POST /api/v1/scanner/sync

POST /api/v1/usdd
```

Use:

- API Resources
- Pagination
- Consistent JSON responses
- Problem-details error format
- API rate limits
- Request IDs
- Idempotency keys
- Sanctum tokens
- Device authentication
- OpenAPI documentation

---

## 45. API RESPONSE FORMAT

Use a consistent response format.

Success:

```
{
  "success": true,
  "message": "Ticket validated successfully.",
  "data": {},
  "meta": {
    "request_id": "..."
  }
}
```

Error:

```
{
  "success": false,
  "message": "The ticket has already been checked in.",
  "error": {
    "code": "TICKET_ALREADY_CHECKED_IN",
    "details": {}
  },
  "meta": {
    "request_id": "..."
  }
}
```

Do not expose stack traces in production.

---

## 46. USER INTERFACE REQUIREMENTS

The interface must look like a professional commercial SaaS product.

Use:

- Clean cards
- Responsive tables
- Mobile-friendly navigation
- Accessible forms
- Clear status badges
- Confirmation dialogs
- Toast notifications
- Loading states
- Empty states
- Error states
- Search and filters
- Consistent spacing
- Clear typography

Do not use a generic uncustomized Bootstrap appearance.

Create:

- Public layout
- Customer layout
- Organizer dashboard layout
- Platform administration layout
- Scanner layout
- Agent POS layout

Use configurable branding.

---

## 47. ACCESSIBILITY

Implement:

- Proper labels
- Keyboard navigation
- Visible focus states
- Semantic HTML
- ARIA only where necessary
- Sufficient contrast

- Screen-reader-friendly validation
- Accessible dialogs
- Accessible ticket selection
- Non-color-only scanner indicators

Scanner results should use:

- Text
  - Icons
  - Sound
  - Vibration
  - Color
- 

## 48. TESTING

Use automated tests extensively.

### Unit tests

Test:

- Fee calculations
- Commission calculations
- Phone normalization
- Ticket-code generation
- QR signature validation
- Inventory calculations
- Ledger balancing
- State transitions
- Settlement calculations
- Discount calculations

### Feature tests

Test:

- Organizer registration
- Organizer approval
- Event creation
- Ticket-type creation
- Checkout
- Payment initiation
- Successful webhook
- Duplicate webhook
- Failed payment

- Ticket issuance
- Ticket retrieval
- Ticket transfer
- Refund
- Promoter attribution
- Agent cash sale
- Settlement
- QR validation
- Duplicate scan
- Offline sync
- USSD purchase

## Security tests

Test:

- Cross-tenant record access
- Unauthorized finance actions
- Forged webhook
- Replay webhook
- QR tampering
- Ticket enumeration
- Brute-force OTP
- Invalid scanner device
- Invalid signed URL

## Concurrency tests

Test:

- Last ticket purchased by multiple users
- Same seat selected simultaneously
- Duplicate successful callbacks
- Simultaneous check-in
- Promo-code usage limit
- Settlement being processed twice

Use factories, seeders and fake providers.

---

## 49. DEMO AND DEVELOPMENT DATA

Create seeders for:

- Platform administrators
- Approved organizer

- Pending organizer
- Sample venue
- General-admission event
- Reserved-seating event
- Ticket types
- Customers
- Promoters
- Agents
- Orders
- Payments
- Tickets
- Gates
- Scanner devices
- Settlements

Use clearly fake data.

Provide local test credentials in the README, but never use those credentials in production.

---

## 50. OBSERVABILITY

Implement:

- Structured logs
- Request correlation IDs
- Queue monitoring
- Failed-job reporting
- Payment webhook monitoring
- Notification delivery monitoring
- Exception tracking integration point
- Health endpoints
- Database health check
- Redis health check
- Queue health check
- Storage health check
- Payment-provider health status

Create an administration page showing:

- Failed jobs
- Failed notifications
- Unprocessed webhooks
- Pending payments
- Queue status
- Scheduler status
- Recent application errors

Mask personal and payment data in logs.

---

## 51. DEPLOYMENT

The platform must be deployable on a Linux VPS.

Provide:

- `.env.example`
- Nginx configuration example
- Queue worker configuration
- Supervisor configuration
- Cron configuration
- Redis configuration notes
- Storage-link instructions
- File permission instructions
- Database migration procedure
- Backup procedure
- Deployment checklist
- Rollback procedure
- SSL requirements
- Production optimization commands
- Horizon configuration
- Log rotation
- Health-check setup

Do not design the production application around shared hosting.

Create a limited fallback document explaining what will not work reliably on shared hosting.

---

## 52. BACKUPS AND DISASTER RECOVERY

Provide procedures for:

- Daily database backups
- Encrypted offsite backups
- Uploaded-file backups
- Point-in-time recovery where available
- Backup retention
- Restore testing
- QR signing-key backup
- Environment-secret backup
- Redis non-authoritative recovery

- Incident recovery

Ticket and financial data must be recoverable.

---

## 53. REQUIRED DOCUMENTATION

Generate:

```
README.md
ARCHITECTURE.md
DATABASE.md
API.md
USSD.md
PAYMENTS.md
SCANNER.md
OFFLINE_SCANNING.md
SECURITY.md
DEPLOYMENT.md
TESTING.md
OPERATIONS.md
DISASTER_RECOVERY.md
```

The README must include:

- Project overview
  - Requirements
  - Local installation
  - Environment variables
  - Database setup
  - Redis setup
  - Queue setup
  - Scheduler setup
  - Running tests
  - Building frontend assets
  - Local payment simulation
  - Local USSD simulation
  - Demo accounts
  - Deployment summary
-

## 54. CODING STANDARDS

Follow:

- PSR-12
- Strict typing where practical
- Typed properties
- Return types
- Dependency injection
- SOLID principles
- Clear domain naming
- Small focused classes
- Form Requests
- Policies
- Value objects for money, phone numbers and references
- Database transactions for critical operations
- No duplicated business logic
- No giant controllers
- No giant service classes
- No raw queries unless justified
- No hardcoded provider credentials
- No hardcoded commission rates
- No silent exception swallowing

Use comments only where they add meaningful context.

Avoid unnecessary abstraction, but keep provider integrations replaceable.

---

## 55. IMPLEMENTATION PHASES

Build the project incrementally.

### Phase 1: Foundation

Implement:

- Laravel project setup
- Authentication
- Roles and permissions
- Organization multi-tenancy
- Organizer onboarding
- KYC foundation
- Settings
- Audit logs

- Core layouts
- Test framework
- Documentation foundation

## **Phase 2: Events and ticket inventory**

Implement:

- Venues
- Events
- Event sessions
- Ticket types
- Inventory
- Reservation engine
- Public event pages
- Event publishing
- General-admission ticket management

## **Phase 3: Checkout and orders**

Implement:

- Guest checkout
- Customer records
- Checkout sessions
- Orders
- Order items
- Fees
- Discounts
- Inventory holds
- Order expiration

## **Phase 4: Payments**

Implement:

- Payment contracts
- Fake payment gateway
- One real provider adapter structure
- Payment attempts
- Webhooks
- Idempotency
- Verification
- Reconciliation
- Payment tests

Use sandbox mode and configurable provider credentials.

## **Phase 5: Ticket issuance and delivery**

Implement:

- Ticket creation
- Secure tokens
- QR generation
- Mobile ticket pages
- PDF tickets
- SMS architecture
- Email delivery
- Ticket retrieval
- Customer ticket wallet

## **Phase 6: USSD**

Implement:

- USSD provider contract
- State-machine flow
- Event-code search
- Ticket selection
- Quantity selection
- Mobile Money initiation
- Payment status
- SMS ticket delivery
- USSD simulator
- Automated USSD tests

## **Phase 7: Scanner and gates**

Implement:

- Gates
- Scanner users
- Device registration
- PWA scanner
- QR scanning
- Manual validation
- Duplicate detection
- Scan reports
- Offline package
- Sync architecture

## Phase 8: Promoters and agents

Implement:

- Promoter profiles
- Referral attribution
- Commission calculations
- Promoter dashboard
- Agent POS
- Cash sales
- Shifts
- Reconciliation

## Phase 9: Refunds, ledger and settlements

Implement:

- Refunds
- Immutable ledger
- Organizer balances
- Platform commission
- Settlement calculations
- Settlement workflow
- Statements
- Reconciliation tools

## Phase 10: Advanced features

Implement:

- Reserved seating
- Ticket transfers
- Local event server interfaces
- Public API
- White-label support
- Custom domains
- Advanced fraud monitoring
- Additional notification providers

---

## 56. REQUIRED WORKFLOW FOR CODEX

Follow these rules while building:

1. Inspect the current repository before modifying anything.

2. Do not assume files exist.
  3. Create a task checklist before starting each phase.
  4. Implement one coherent phase at a time.
  5. Run migrations after creating them.
  6. Run relevant automated tests after each module.
  7. Run the complete test suite before completing a phase.
  8. Fix all failing tests before proceeding.
  9. Run code formatting.
  10. Run static analysis if configured.
  11. Update documentation during every phase.
  12. Do not leave placeholder methods that silently return success.
  13. Do not use fake payment success in production code.
  14. Do not hardcode credentials.
  15. Do not delete working functionality without explaining why.
  16. Preserve backward compatibility where reasonable.
  17. Explain database migrations that may affect existing data.
  18. Report any architectural decision that changes the specification.
  19. Use clear commit-sized changes.
  20. At the end of each phase, provide:
    21. Files created
    22. Files modified
    23. Migrations added
    24. Routes added
    25. Tests added
    26. Commands to run
    27. Manual testing procedure
    28. Remaining work
- 

## 57. FIRST TASK

Begin with Phase 1 only.

Before writing code:

1. Inspect the repository.
2. Report the current framework version, PHP requirements and existing structure.
3. Identify existing packages and conventions.
4. Propose the exact Phase 1 file and database structure.
5. Identify any conflicts with this specification.
6. Create a Phase 1 implementation checklist.

Then implement Phase 1.

Phase 1 must deliver:

- Base application configuration
- Responsive public layout
- Responsive platform-admin layout
- Responsive organizer layout
- Authentication
- Customer phone-number model foundation
- Roles and permissions
- Organization multi-tenancy
- Organization registration
- Organization team members
- Organizer KYC foundation
- Platform administration
- Settings foundation
- Audit logging
- Policies
- Seeders
- Factories
- Automated tests
- Documentation

Do not begin payment, ticket issuance, USSD or scanner functionality until Phase 1 is tested and stable.

---

## 58. DEFINITION OF DONE FOR PHASE 1

Phase 1 is complete only when:

- Fresh installation works.
- Migrations run successfully.
- Seeders run successfully.
- Authentication works.
- Platform admin can log in.
- Organizer can register.
- Platform admin can review an organizer.
- Approved organizer can access its dashboard.
- Suspended organizer cannot access protected organizer functions.
- Organizer can invite a team member.
- Roles and permissions are enforced.
- Cross-tenant access tests pass.
- KYC documents can be submitted securely.
- Audit records are created for important actions.
- Responsive layouts work on desktop and mobile.
- All Phase 1 tests pass.
- README is updated.

- No sensitive credentials are committed.
- No critical TODO placeholders remain.

Start now with repository inspection and the Phase 1 checklist.